

INSTITUTO DE INFORMÁTICA — UFG

Introdução à Engenharia de Software

2026/1

Construção de Software com Métodos Ágeis e Testes Focados em UX

Profa. Dra. Renata Dutra Braga

renatadbraga@ufg.br

INSTITUTO DE
INFORMÁTICA - INF



UNIVERSIDADE FEDERAL DE GOIÁS

Objetivos da Aula

Ao final desta aula, você será capaz de compreender como times ágeis trabalham, por que UX é parte essencial do desenvolvimento e como os testes garantem qualidade na prática.

1

Métodos Ágeis

Entender o que são e como organizam o desenvolvimento de software

2

UX no Desenvolvimento

Relacionar experiência do usuário com cada etapa do projeto

3

Testes Focados no Usuário

Compreender os tipos de testes e como protegem a qualidade

4

Projeto Real

Aplicar os conceitos no sistema de gestão do Shopping Flamboyant

Síntese

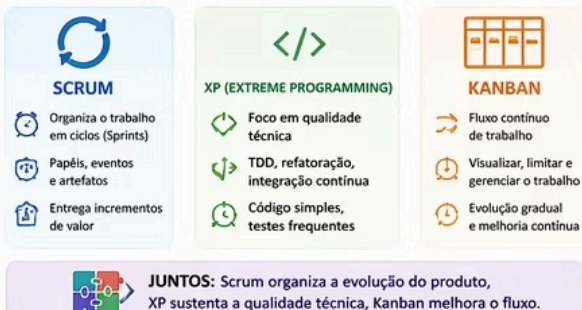
Mais detalhes, acesse a wiki no repositório da nossa disciplina ([Aula 8](#)).

CONSTRUÇÃO DE SOFTWARE COM MÉTODOS ÁGEIS E TESTES FOCADOS EM REQUISITOS DE UX

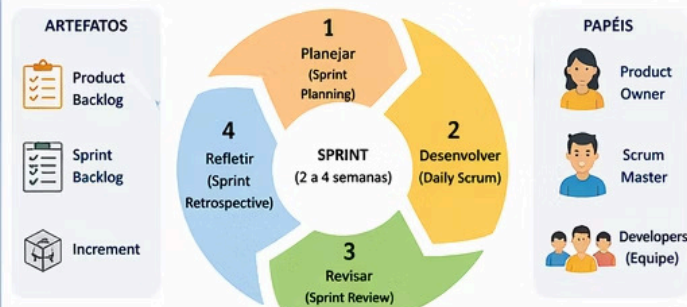
1. VISÃO GERAL



2. MÉTODOS ÁGEIS: COMO FUNCIONAM JUNTOS



3. CICLO ÁGIL (SCRUM)



4. ENGENHARIA DE REQUISITOS ÁGIL E UX



5. PIRÂMIDE DE TESTES COM FOCO EM UX



6. TESTES: DO FUNCIONAL AO UX



7. APLICAÇÃO NO SHOPPING FLAMBOYANT



8. BACKLOG INICIAL DO PRODUTO



9. EXEMPLO DE SPRINT DIDÁTICA



10. DECISÕES DE UX NO SISTEMA



Por Que Desenvolver Software É Difícil?

O Problema

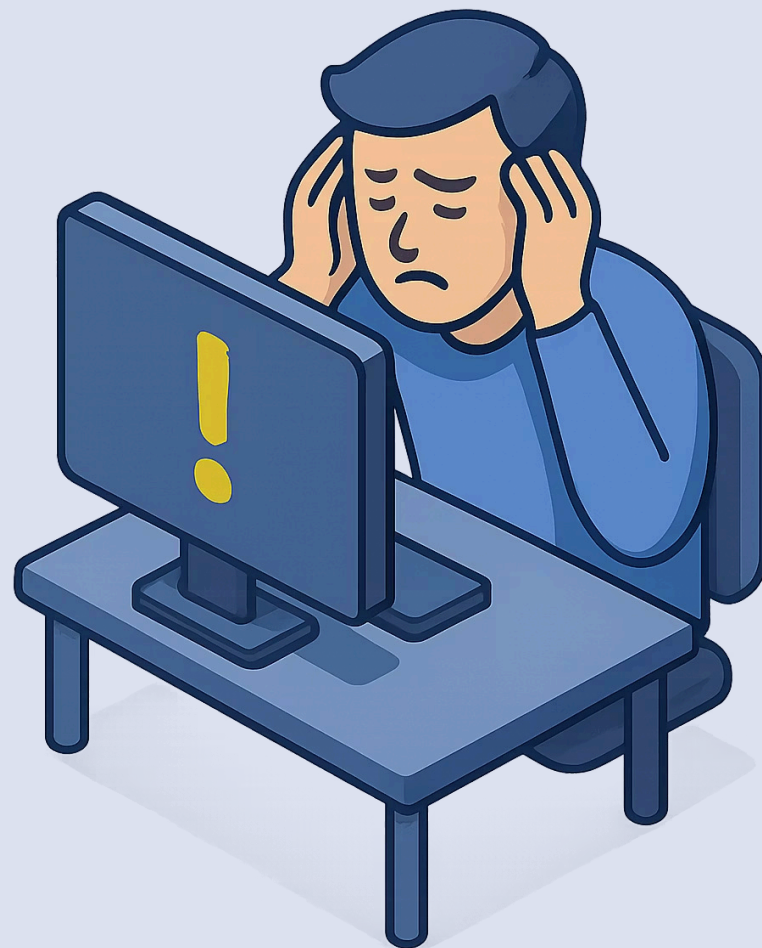
Muitos sistemas são tecnicamente funcionais, mas ninguém consegue usá-los. O usuário se perde, não encontra o que precisa, e abandona o sistema.

- Interfaces confusas e pouco intuitivas
- Funcionalidades que não correspondem à necessidade real
- Falta de feedback durante o uso

A Solução

Desenvolvimento de software eficaz une **boas práticas de engenharia** com **foco no usuário desde o primeiro dia**. Métodos ágeis e UX caminham juntos para resolver esse problema.

- ❗ UX não é etapa final — é ponto de partida.



O Que São Métodos Ágeis?

Métodos ágeis são abordagens de desenvolvimento que substituem o planejamento rígido por **entregas frequentes, ciclos curtos e adaptação constante**. Em vez de entregar tudo no final, entregamos partes funcionais ao longo do projeto.



Desenvolvimento Incremental

O sistema cresce em partes pequenas e funcionais, chamadas incrementos.



Entregas Frequentes

A cada ciclo (sprint), algo novo e utilizável é entregue ao cliente.



Feedback Constante

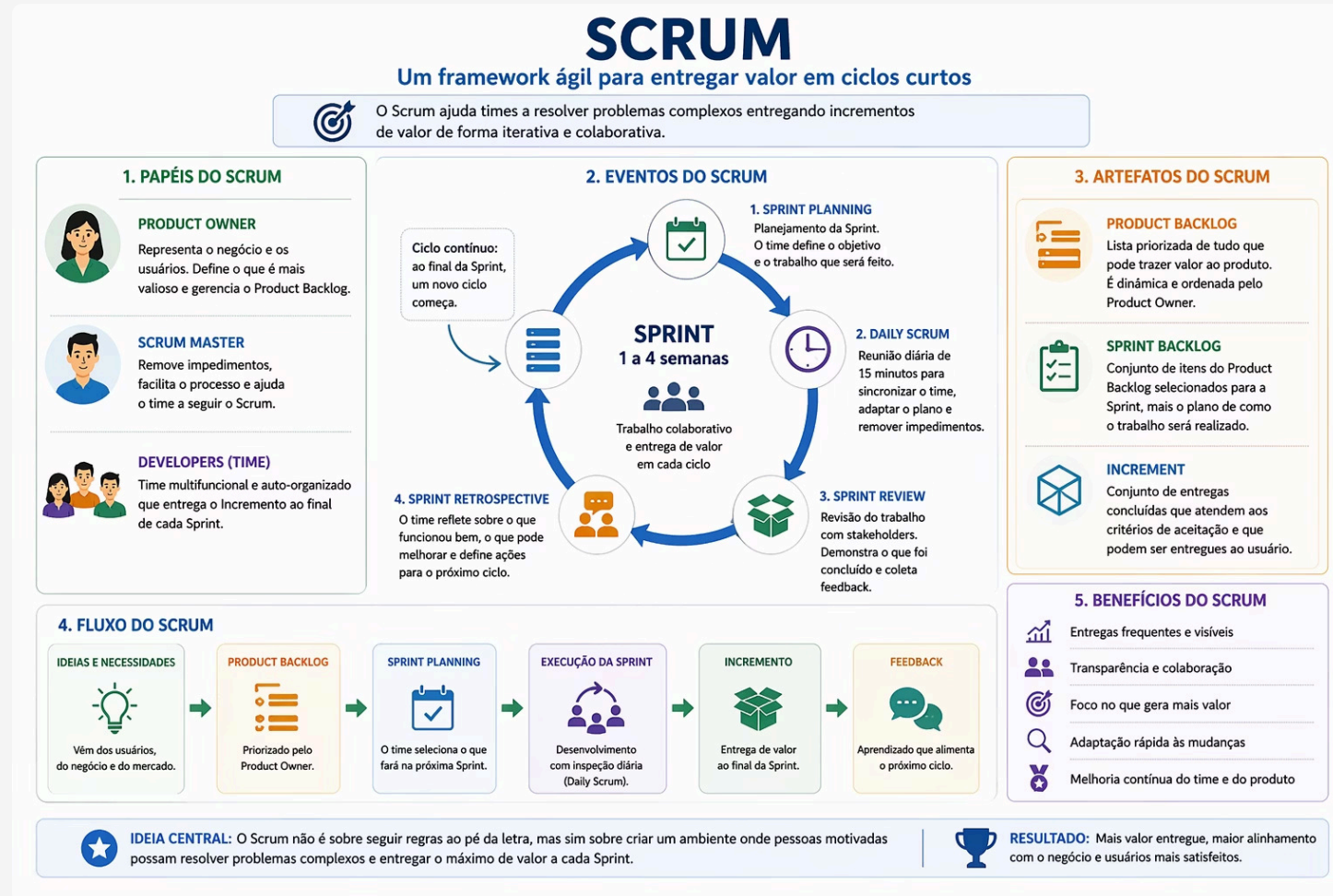
O cliente participa ativamente, avaliando e sugerindo ajustes a cada entrega.



Colaboração

A equipe e o cliente trabalham juntos, não de forma isolada.

Scrum: O Framework Mais Usado



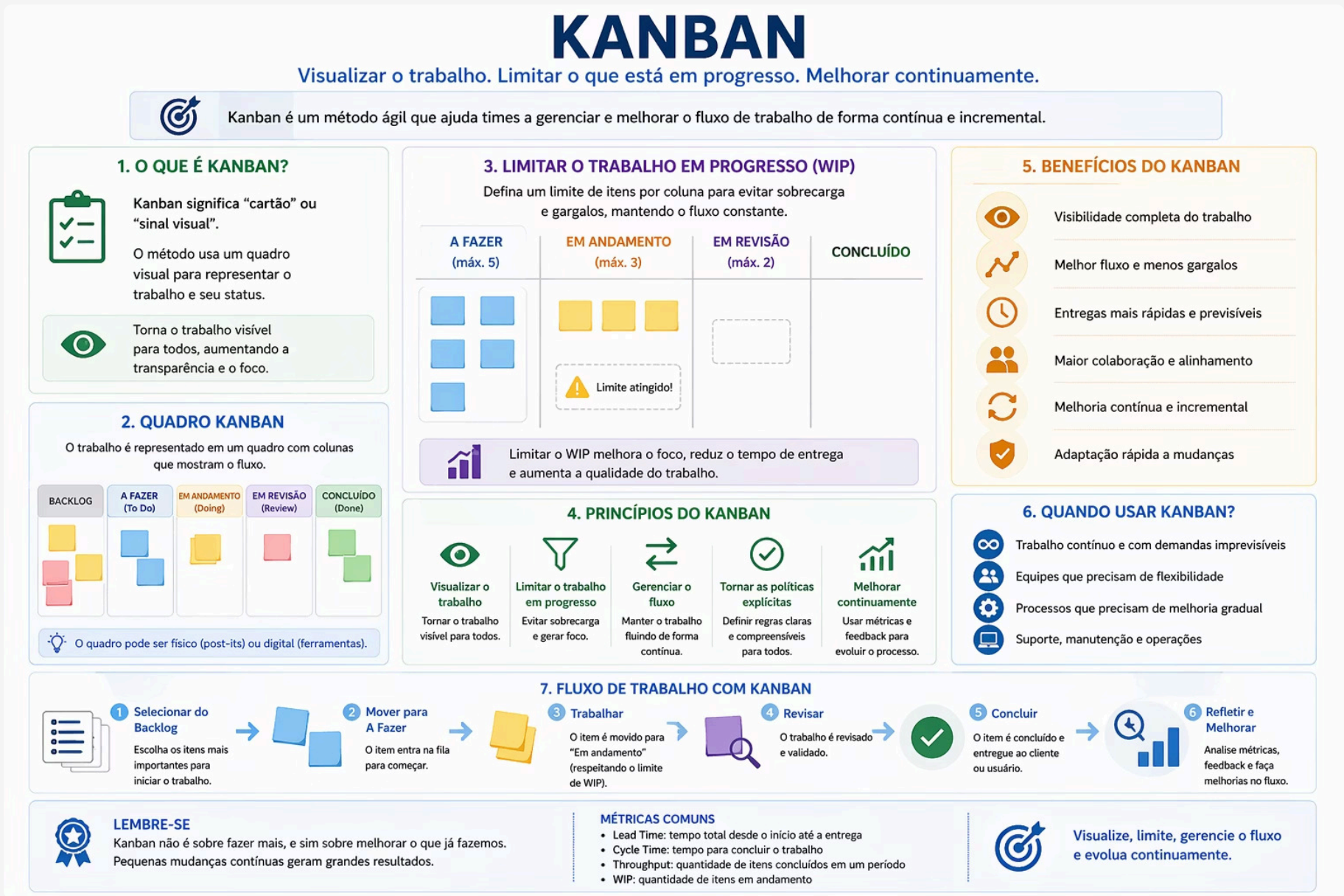
Cerimônias do Scrum

- **Planning:** planejamento do que será feito na sprint
- **Daily:** reunião diária de 15 minutos (o que fiz, o que farei, impedimentos)
- **Review:** demonstração do que foi entregue
- **Retrospectiva:** melhoria contínua do processo

❏ Uma **Sprint** dura de 1 a 4 semanas e termina sempre com algo funcionando.

Kanban: Visualizando o Fluxo de Trabalho

O Kanban organiza as tarefas em um quadro visual dividido em colunas. Cada tarefa é um cartão que avança conforme o trabalho progride. É simples, visual e muito eficaz para controlar o que está acontecendo.



O Kanban é ideal para equipes que têm demanda contínua e variada, sem ciclos fixos como o Scrum. Limitar o número de tarefas em andamento (**WIP limit**) evita sobrecarga e melhora o foco.

XP — Extreme Programming

O XP é um método ágil que enfatiza **qualidade técnica** e boas práticas de codificação. Funciona bem em projetos onde os requisitos mudam com frequência.

XP – EXTREME PROGRAMMING

Qualidade de software e capacidade de resposta a mudanças



XP é uma abordagem ágil que combina práticas de engenharia de software com valores como comunicação, simplicidade, feedback constante e respeito às pessoas.

VALORES DO XP



Comunicação
Todos se comunicam de forma clara e frequente.



Simplicidade
Fazer o que é necessário, nada além disso.



Feedback
Ciclos curtos para aprender e ajustar.



Coragem
Ter disposição para tomar decisões e mudar.



Respeito
Valorizar as pessoas e o trabalho em equipe.

1. PRINCIPAIS PRÁTICAS DO XP



Programação em par
Duas pessoas trabalham juntas no mesmo código.



Test-Driven Development (TDD)
Escrever testes antes do código. Código só é escrito para passar no teste.



Integração Contínua
Integrar o código várias vezes ao dia para detectar problemas cedo.



Refatoração
Melhorar o código continuamente sem mudar seu comportamento.



Design Simples
Escolher a solução mais simples que atenda ao momento atual.



Padrões de Código
Seguir convenções para manter o código consistente.



Propriedade Coletiva do Código
Qualquer pessoa do time pode alterar qualquer parte do código.



Ritmo Sustentável
Manter um ritmo constante para evitar sobrecarga.



Cliente Presente
O cliente participa ativamente e está disponível para o time.

2. CICLO DO XP



3. COMO FUNCIONA O TDD NO XP



★ TDD garante código testável, simples e com menos defeitos.

4. XP NA PRÁTICA



5. BENEFÍCIOS DO XP

- Alta qualidade de código**
Menos defeitos e mais confiança nas entregas.
- Responde bem a mudanças**
Fácil de adaptar quando requisitos mudam.
- Entregas frequentes**
Valor entregue em ciclos curtos.
- Maior colaboração**
Comunicação constante e trabalho em equipe.
- Código simples e sustentável**
Mais fácil de entender, manter e evoluir.

6. QUANDO USAR XP?

- Projetos com requisitos que mudam com frequência
- Times pequenos ou médios com alto nível de colaboração
- Necessidade de alta qualidade e tecnologia
- Entregas rápidas e contínuas de valor



LEMBRE-SE

XP não é sobre ferramentas, e sim sobre disciplina, boas práticas e foco em entregar o que realmente importa para o usuário.



OBJETIVO FINAL DO XP

Satisfazer o cliente entregando software de alta qualidade, com simplicidade, frequência e adaptabilidade.
Menos é mais. Feedback é poder. Qualidade é prioridade.

Engenharia de Requisitos no Contexto Ágil

Em projetos ágeis, os requisitos não são documentos extensos e imutáveis. Eles evoluem ao longo do projeto, organizados em formato acessível e priorizado.

User Stories

Descrições curtas de funcionalidades do ponto de vista do usuário. São a forma principal de capturar o que o sistema deve fazer.

Product Backlog

Lista ordenada de todas as funcionalidades desejadas. O **Product Owner** decide a prioridade com base no valor para o cliente.

Critérios de Aceitação

Condições que a funcionalidade precisa atender para ser considerada pronta. Definem o que será testado.

Como Escrever uma User Story

Estrutura padrão:

"Como [tipo de usuário], quero [realizar uma ação], para [atingir um objetivo]."

Exemplo 1 — Lojista

"Como **lojista**, quero **registrar um atendimento**, para **manter o histórico de interações com o shopping**."

Exemplo 2 — Gerente

"Como **gerente**, quero **visualizar um dashboard**, para **acompanhar os indicadores das lojas em tempo real**."

Exemplo 3 — Analista

"Como **analista**, quero **filtrar atendimentos por data**, para **gerar relatórios mensais com facilidade**."

 User stories bem escritas facilitam a comunicação entre time técnico e cliente — e se tornam a base dos testes.

Integrando UX ao Processo Ágil

UX (User Experience) não é uma etapa que acontece no final do projeto. Em times ágeis, o design centrado no usuário acontece **em paralelo ao desenvolvimento**, em cada sprint.



O Que É UX — Experiência do Usuário

UX é como uma pessoa se sente ao usar um sistema. Não se trata apenas de "ser bonito" — envolve **facilidade, eficiência, clareza e satisfação**.

- ✔ Um sistema com boa UX é aquele que o usuário consegue usar sem precisar de treinamento.

Exemplo prático: Um app de banco que exige 7 cliques para fazer um PIX tem UX ruim. Um que faz em 2 cliques tem UX boa.



Eficiência

O usuário consegue fazer o que precisa rapidamente



Satisfação

A experiência é agradável e sem frustração



Acessibilidade

Funciona para todos, incluindo pessoas com deficiência

Testes de Software em Projetos Ágeis

Em metodologias ágeis, testar não é uma fase separada — é uma atividade contínua. A cada nova funcionalidade, os testes garantem que o sistema continua funcionando corretamente.

1

Escreva o Teste

Defina o que a funcionalidade deve fazer antes de construí-la

2

Implemente

Desenvolva o código para fazer o teste passar

3

Automatize

Automatize testes para rodar a cada nova mudança no código

4

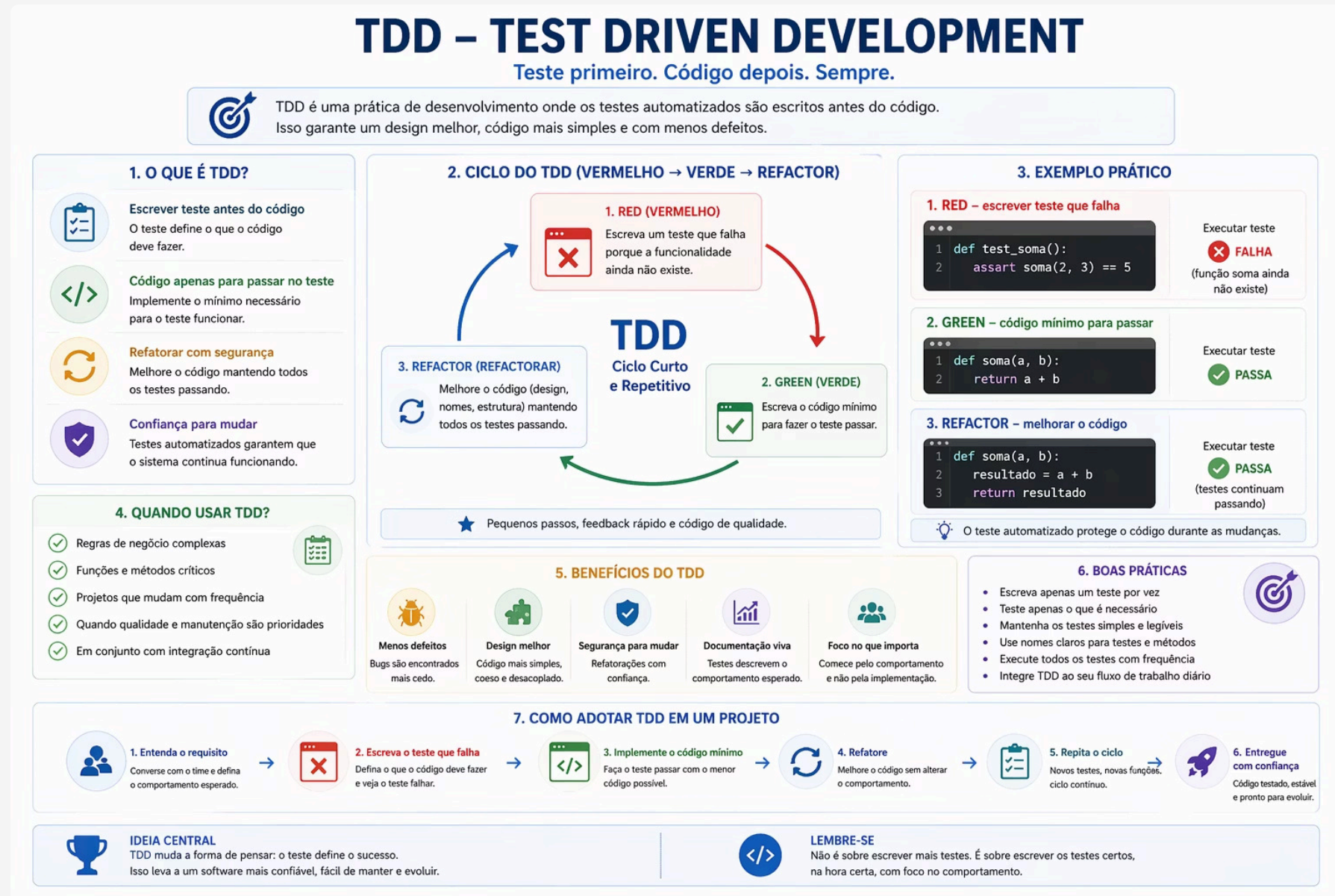
Melhore

Refatore o código mantendo todos os testes passando

⚠️ Um código sem testes é um código que você não confia completamente.

TDD — Test Driven Development

No TDD, o teste é escrito **antes** do código. Isso parece estranho no início, mas garante que o código seja escrito apenas para resolver um problema real e testável.



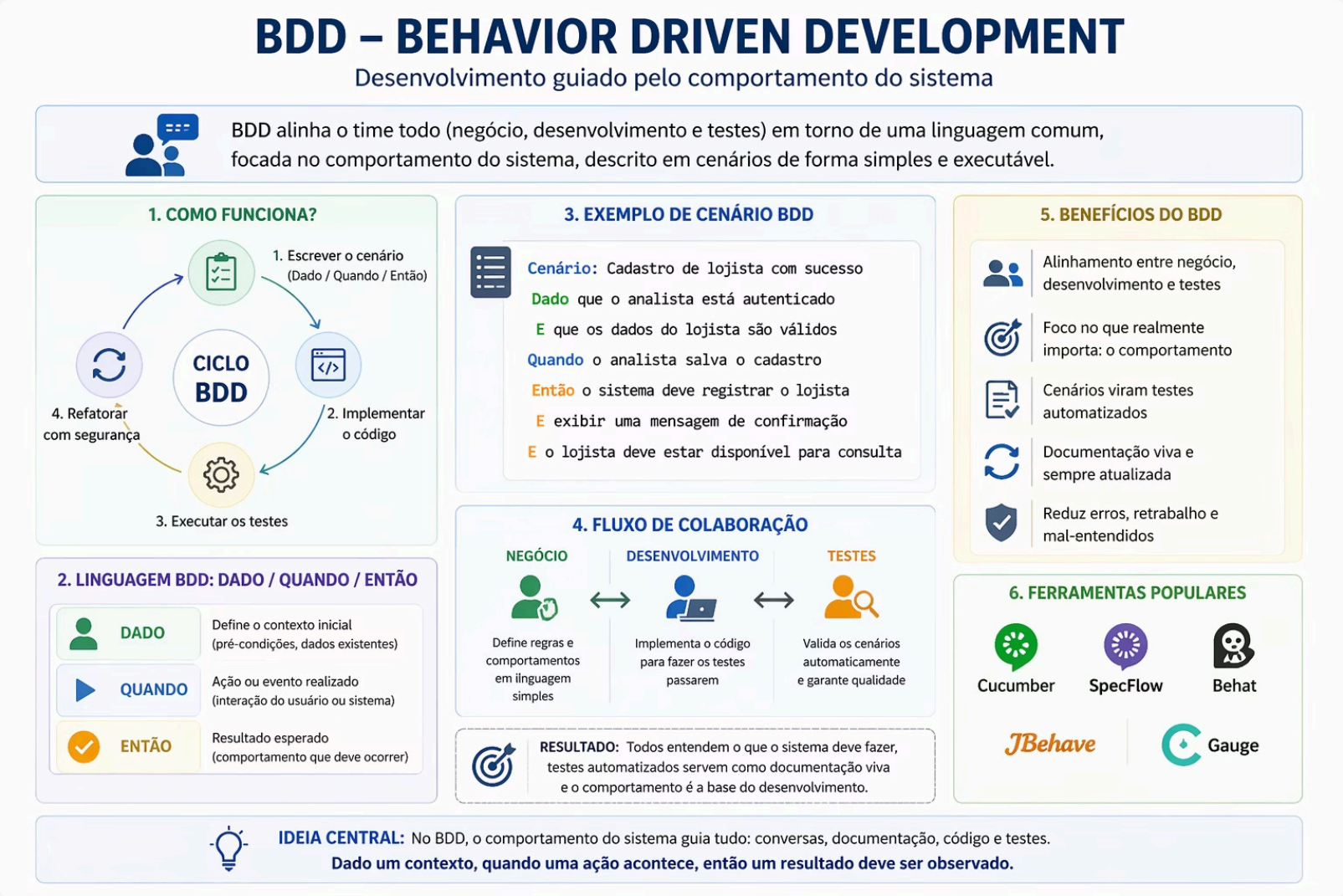
Por que usar TDD?

- Força o desenvolvedor a pensar no comportamento esperado antes de codificar
- Reduz a quantidade de bugs
- Facilita refatoração com segurança

  O ciclo Red → Green → Refactor é o coração do TDD. Repita para cada nova funcionalidade.

BDD — Behavior Driven Development

O BDD amplia o TDD com uma linguagem que tanto desenvolvedores quanto clientes entendem. Os cenários descrevem o **comportamento esperado do sistema** em linguagem natural estruturada.



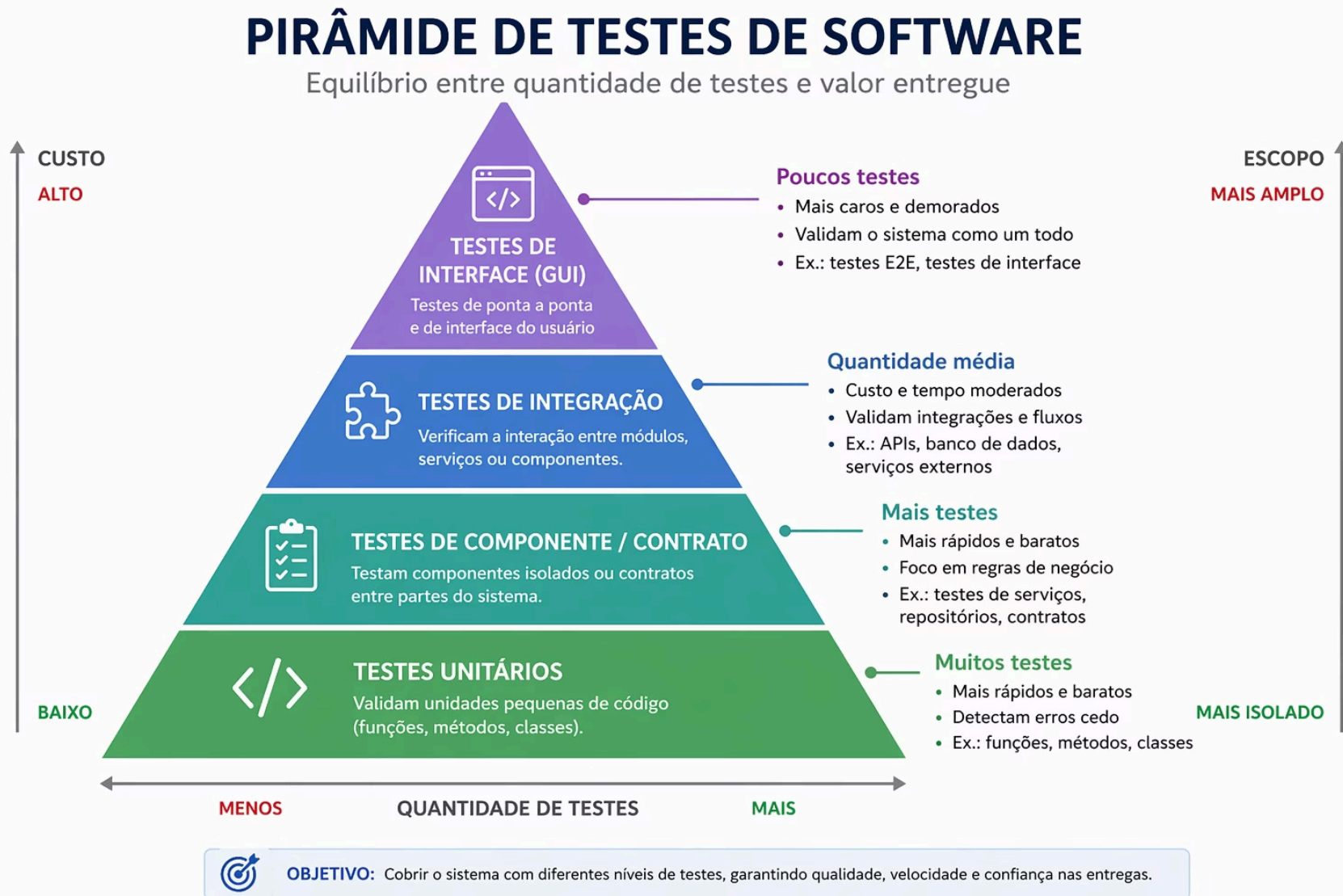
Estrutura Dado / Quando / Então:

- Dado que o usuário está logado no sistema
- Quando ele busca o nome de um lojista
- Então o sistema deve exibir o histórico completo de atendimentos

Dado (Given)	Quando (When)	Então (Then)
O contexto inicial — o estado do sistema antes da ação	A ação realizada pelo usuário ou pelo sistema	O resultado esperado — o que o sistema deve fazer

A Pirâmide de Testes

A pirâmide de testes mostra como equilibrar os diferentes tipos de teste. A base deve ser larga (muitos testes unitários, rápidos e baratos) e o topo estreito (menos testes de interface, lentos e caros).



  Regra prática: 70% unitários · 20% integração · 10% interface.

Testes Focados em UX

Além dos testes técnicos, sistemas precisam ser validados com **usuários reais**. Esses testes garantem que a interface é compreensível, acessível e satisfatória.



Teste de Usabilidade

Um usuário real realiza tarefas no sistema enquanto é observado. Revelar dificuldades que o time não percebe sozinho.



Teste de Acessibilidade

Verifica se o sistema funciona para pessoas com deficiência visual, motora ou cognitiva. Ferramentas como leitores de tela são usadas.



Avaliação de Satisfação

Questionários (como o SUS — System Usability Scale) medem o quanto o usuário ficou satisfeito com a experiência.



Contexto: Sistema do Shopping Flamboyant

O Cenário

O Shopping Flamboyant precisa de um sistema para gerenciar lojistas, registrar atendimentos e gerar relatórios analíticos para a administração.

Problemas Identificados

- Dados dos lojistas espalhados em planilhas diferentes
- Histórico de atendimentos sem registro unificado
- Dificuldade para analisar indicadores por período ou loja
- Processo manual e sujeito a erros

⚠ Esses problemas afetam diretamente a UX dos funcionários que usam o sistema diariamente.



User Stories do Projeto Flamboyant

A equipe levantou as principais necessidades do sistema e transformou em user stories priorizadas no backlog:

1

Cadastro de Lojista

"Como **analista**, quero cadastrar novos lojistas, para manter a base de dados atualizada e centralizada."

2

Registro de Atendimento

"Como **atendente**, quero registrar atendimentos por loja, para guardar o histórico de interações."

3

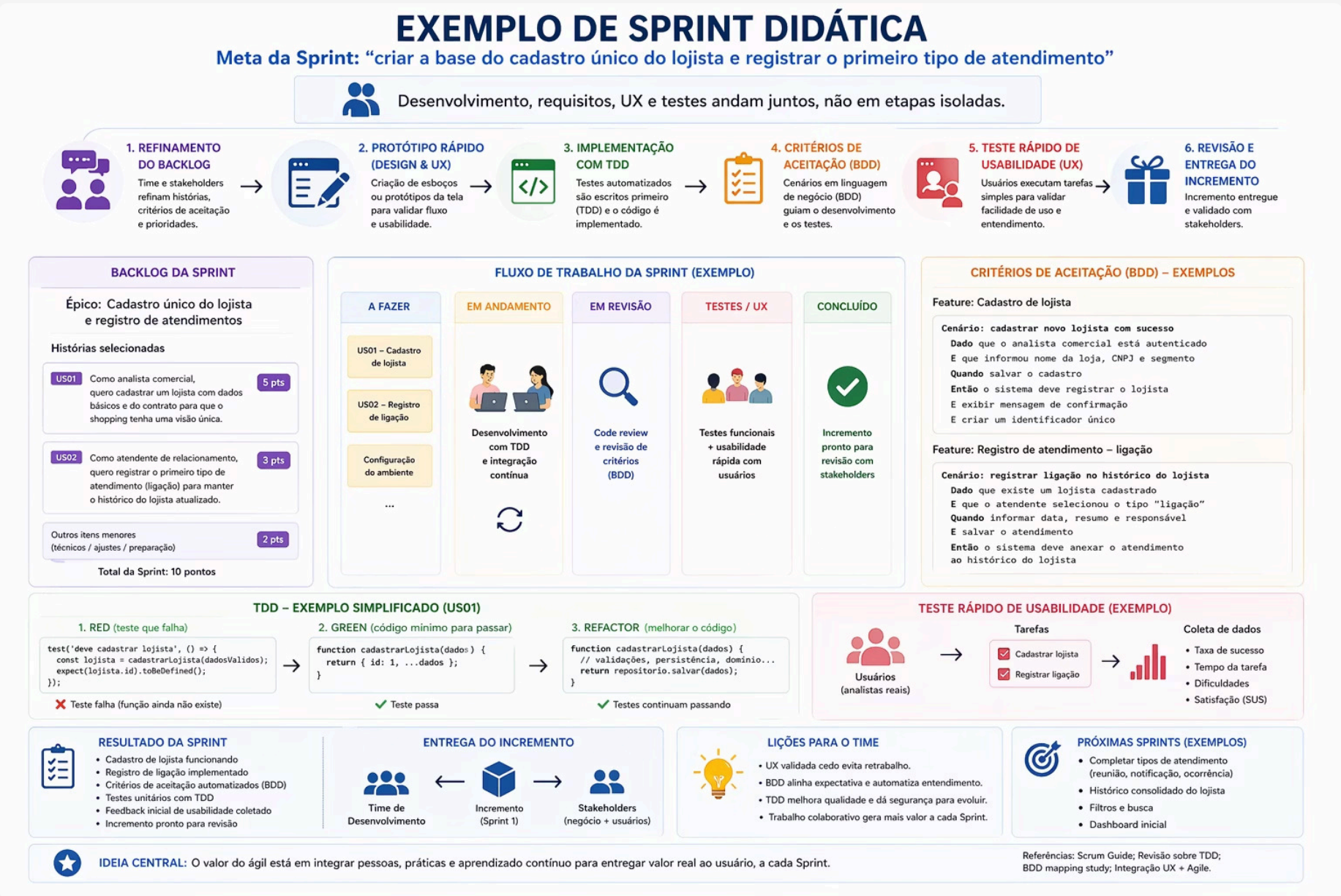
Consulta de Histórico

"Como **gerente**, quero consultar o histórico de um lojista, para entender o relacionamento ao longo do tempo."

4

Dashboard Analítico

"Como **diretor**, quero visualizar um dashboard com filtros, para tomar decisões baseadas em dados."



Critérios de Aceitação e Testes no Flamboyant

Cenário BDD — Consulta de Histórico

Dado que o usuário está autenticado no sistema

Quando ele busca pelo nome de um lojista

Então o sistema exibe o histórico completo de atendimentos
ordenado por data

Tipos de Testes Aplicados

- **Funcional**

Verifica se o sistema registra
e retorna os dados
corretamente

- **Usabilidade**

Testa se o atendente
consegue usar o sistema sem
dificuldade

- **UX / Satisfação**

Mede se a interface é intuitiva e agradável no dia a dia

Resumo: O Que Aprendemos Hoje

Desenvolvemos software melhor quando combinamos **organização ágil, foco no usuário e testes contínuos**. Esses três pilares formam a base de um bom engenheiro de software moderno.



Métodos Ágeis

Organizam o trabalho em ciclos curtos, com entregas constantes e adaptação ao feedback



UX desde o Início

A experiência do usuário deve guiar cada decisão de design e desenvolvimento



Testes Garantem Qualidade

TDD, BDD e testes de usabilidade asseguram que o sistema funciona e agrada quem usa

❓ 🎓 **Para reflexão:** Por que UX deve ser considerado desde o início? Qual a vantagem real de escrever testes antes do código?

Definindo Critérios de Aceitação

É hora de aplicar o conhecimento! Nesta atividade, você consolidará os critérios de aceitação para as user stories do projeto Flamboyant, usando a estrutura BDD e registrando-os em uma plataforma colaborativa.

Pacote 3 – Área de Testes de Software: 3º produto: Critérios de aceitação (DADO, QUANDO, ENTÃO) Registro em markdown (wiki) no GitHub.

1

Dado, Quando, Então

Para cada user story do projeto, defina os cenários de uso com a estrutura BDD (Dado, Quando, Então).

2

Registro em Markdown

Documente esses critérios de aceitação em formato Markdown, aproveitando sua simplicidade e clareza.

3

Publicação no GitHub Wiki

Compartilhe os critérios no GitHub Wiki do projeto, garantindo que toda a equipe tenha acesso e possa colaborar.

